Using SelfControlledCohort

2026-03-26

Contents

The **SelfControlledCohort** package estimates the association between an exposure and an outcome by comparing the rate of outcomes during exposed time to the rate of outcomes during unexposed time within the same cohort of people. It is "self-controlled" because each person serves as their own control.

This vignette demonstrates how to run a Self-Controlled Cohort (SCC) analysis using the `Eunomia` example dataset.

1 Setup and Data

First, we need to load the necessary packages and set up a connection to the `Eunomia` database. `Eunomia` provides a small, simulated Common Data Model (CDM) database for demonstration purposes.

```
library(SelfControlledCohort)
library(Eunomia)
library(DatabaseConnector)
library(dplyr)

# Create a temporary SQLite database with Eunomia data
connectionDetails <- getEunomiaConnectionDetails()
connection <- connect(connectionDetails)

# We will use the 'main' schema for this example
cdmDatabaseSchema <- "main"
```

2 Running a Basic Analysis

For a first example, let's run an analysis across **all drugs** (exposure) and **all conditions** (outcome) available in the database.

We specify: * `exposureTable` / `outcomeTable`: We use the standard CDM tables `drug_era` and `condition_era`. * `exposureIds`: Left empty to include all drugs. * `outcomeIds`: Left empty to include all conditions.

```
# Define a temporary folder for results
outputFolder <- tempfile("scc_output")
dir.create(outputFolder)

# Run the analysis
runSelfControlledCohort(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = cdmDatabaseSchema,
  exposureTable = "drug_era",
  outcomeTable = "condition_era",
  resultExportPath = outputFolder,
  databaseId = 1,
  analysisId = 1
)
#> Computing time at risk exposed and unexposed windows
#>   |                                                                      |
#> Retrieving counts from database
#>   |                                                                      |
#> Computing raw effect estimates
#> Initiating cluster with 7 threads
#> Stopping cluster
#> Running diagnostics
#> Running SCC diagnostics
#> - Running MDRR (power) diagnostic
#> - Running pre-exposure gain diagnostic
#> - Running event-dependent observation diagnostic
#> - Running EASE (systematic error) diagnostic
#> Completed 33248 diagnostic tests
#> 6977 diagnostic test(s) failed:
#> Computing time at risk distribution statistics
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#> Cleaning up intermedate tables
#>   |                                                                      |
#> Performing SCC analysis took 21.3 secs
```

Now we can read the results and inspect the estimates.

```
results <- read.csv(file.path(outputFolder, "scc_result.csv"))

# Display the top 5 associations by Relative Risk
results %>%
  filter(analysis_id == 1) %>%
  arrange(desc(rr)) %>%
  head(5) %>%
  select(target_cohort_id, outcome_cohort_id, rr, lb_95, ub_95, p_value) %>%
  knitr::kable(digits = 3)
```

target_cohort_id	outcome_cohort_id	rr	lb_95	ub_95	p_value
701322	4155034	Inf	0.027	Inf	NA
705944	0	Inf	0.024	Inf	NA
705944	440448	Inf	0.024	Inf	NA
705944	4001336	Inf	0.024	Inf	NA
705944	4048695	Inf	0.024	Inf	NA

The `rr` column shows the Incidence Rate Ratio. An IRR > 1 indicates an increased risk.

3 Understanding Risk Windows

To perform the comparison, each person's observation time is partitioned into "exposed" and "unexposed" risk windows relative to their exposure start and end dates.

You control these windows using the `runSelfControlledCohort()` arguments:

- **Exposed time:** Defined by `riskWindowStartExposed` and `riskWindowEndExposed`. If `addLengthOfExposureExposed = TRUE`, the end window is added to the exposure *end* date instead of the start date.
- **Unexposed time:** Defined by `riskWindowStartUnexposed` and `riskWindowEndUnexposed`. These are typically set to negative values (e.g., `-30` to `-1`) to create a pre-exposure baseline window.

4 Using Custom Cohorts

In practice, we often want to study specific cohorts defined by complex logic (e.g., "New users of NSAIDs"). We can create these cohorts and pass them to the function.

`Eunomia` includes definitions for NSAIDs and GI Bleed. Let's create them.

```
# Create standard cohorts in the 'cohort' table:
# 1 = Celecoxib
# 2 = Diclofenac
# 3 = GI Bleed
# 4 = NSAIDs
createCohorts(connectionDetails)
#> Cohorts created in table main.cohort
#>   cohortId       name
#> 1        1  Celecoxib
#> 2        2 Diclofenac
#> 3        3    GiBleed
#> 4        4     NSAIDs
#>                                                                                  description
#> 1    A simplified cohort definition for new users of celecoxib, designed specifically for Eunomia.
#> 2    A simplified cohort definition for new users ofdiclofenac, designed specifically for Eunomia.
#> 3 A simplified cohort definition for gastrointestinal bleeding, designed specifically for Eunomia.
#> 4       A simplified cohort definition for new users of NSAIDs, designed specifically for Eunomia.
#>   count
#> 1  1844
#> 2   850
#> 3   479
#> 4  2694
```

Now run the analysis using these specific cohorts.

```
runSelfControlledCohort(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = cdmDatabaseSchema,
  exposureTable = "cohort",
  outcomeTable = "cohort",
  exposureIds = c(1, 2, 4), # Celecoxib, Diclofenac, NSAIDs
  outcomeIds = c(3), # GI Bleed
  databaseId = 1,
  analysisId = 2, # Unique ID for this run
  resultExportPath = outputFolder
)
#> Computing time at risk exposed and unexposed windows
#>   |                                                                      |
#> Retrieving counts from database
#>   |                                                                      |
#> Computing raw effect estimates
#> Initiating cluster with 7 threads
#> Stopping cluster
#> Running diagnostics
#> Running SCC diagnostics
#> - Running MDRR (power) diagnostic
#> - Running pre-exposure gain diagnostic
#> - Running event-dependent observation diagnostic
#> - Running EASE (systematic error) diagnostic
#> Completed 9 diagnostic tests
#> 3 diagnostic test(s) failed:
#> Computing time at risk distribution statistics
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#> Cleaning up intermedate tables
#>   |                                                                      |
#> Performing SCC analysis took 2.16 secs
```

```
results <- read.csv(file.path(outputFolder, "scc_result.csv"))

results %>%
  filter(analysis_id == 2) %>%
  select(target_cohort_id, outcome_cohort_id, rr, p_value) %>%
  knitr::kable(digits = 3)
```

target_cohort_id	outcome_cohort_id	rr	p_value
1	3	Inf	NA
2	3	Inf	NA
4	3	Inf	NA

5 Diagnostics and Tiered Blinding

Reliable evidence requires that the analysis assumptions are met. The package evaluates four diagnostics (aligned with the Self-Controlled Case Series package) to check for issues like lack of statistical power or

confounding. Diagnostics are run by default when `runDiagnostics = TRUE`.

The default thresholds can be viewed or modified:

```
thresholds <- getDefaultDiagnosticThresholds()
str(thresholds)
#> List of 6
#>  $ mdrrMaxAcceptable         : num 10
#>  $ maxPreExposureProportion  : num 0.05
#>  $ preExposurePThreshold     : num 0.05
#>  $ maxEventDependentCensoring: num 0.1
#>  $ minEventsPerWindow        : num 3
#>  $ easeMaxAcceptable         : num 0.25
```

We implement a **Tiered Blinding** system based on these diagnostics: 1. **Tier 1 (Unblind for Calibration)**: If core diagnostics pass (Pre-Exposure Gain, Event-Dependent Observation, and EASE) but power (MDRR) is insufficient, the result can still be used as a negative control for calibration. 2. **Tier 2 (Unblind)**: If *all* diagnostics (including power) pass, the result is considered valid for unblinding.

Let's look at the raw diagnostic results:

```
diagnostics <- read.csv(file.path(outputFolder, "scc_diagnostics_summary.csv"))

# Show the diagnostic results for analysis 2
diagnostics %>%
  filter(analysis_id == 2) %>%
  select(target_cohort_id, outcome_cohort_id, diagnostic_name, diagnostic_value, pass) %>%
  head(10) %>%
  knitr::kable()
```

target_cohort_id	outcome_cohort_id	diagnostic_name	diagnostic_value	pass
1	3	MDRR	NA	0
2	3	MDRR	NA	0
4	3	MDRR	NA	0
1	3	UNBLIND	NA	0
2	3	UNBLIND	NA	0
4	3	UNBLIND	NA	0
1	3	UNBLIND_FOR_CALIBRATION	NA	1
2	3	UNBLIND_FOR_CALIBRATION	NA	1
4	3	UNBLIND_FOR_CALIBRATION	NA	1

If the `pass` column is 0, the diagnostic failed. *See the **Study Diagnostics** vignette for a detailed explanation of each test.*

Note: Blinding rules (`UNBLIND` and `UNBLIND_FOR_CALIBRATION`) can be computed from this test data using the `getDiagnosticsSummary()` function on the returned `.runSccDiagnostics` object in memory, or by applying the tiered blinding logic to the CSV table manually.

6 Empirical Calibration

Observational studies often suffer from systematic error (bias). We can use **Empirical Calibration** to adjust for this bias using negative controls—pairs of exposures and outcomes where we believe the true effect size is 1 (no effect).

In this example, we will treat **Celecoxib (1)** and **Diclofenac (2)** combined with **GI Bleed (3)** as "negative controls" purely for demonstration purposes.

We supply the `negativeControlPairs` argument. These pairs are used to fit a null distribution which is then used to calibrate the p-values and confidence intervals of our target analysis (**NSAIDs (4)** vs **GI Bleed (3)**).

Note: When negative controls are supplied, the package also calculates the Expected Absolute Systematic Error (EASE) diagnostic.

```
# Define negative controls (formatted as list of vectors: c(exposureId, outcomeId))
negativeControls <- list(
  c(1, 3), # Celecoxib - GI Bleed (Demo only!)
  c(2, 3) # Diclofenac - GI Bleed (Demo only!)
)

runSelfControlledCohort(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = cdmDatabaseSchema,
  exposureTable = "cohort",
  outcomeTable = "cohort",
  exposureIds = c(4), # Target: NSAIDs
  outcomeIds = c(3), # Outcome: GI Bleed
  analysisId = 4, # Unique ID for this calibration run
  negativeControlPairs = negativeControls, # <--- Supply negative controls
  resultExportPath = outputFolder,
  databaseId = 1
)
#> Computing time at risk exposed and unexposed windows
#>   |                                                                      |
#> Retrieving counts from database
#>   |                                                                      |
#> Computing raw effect estimates
#> Initiating cluster with 7 threads
#> Stopping cluster
#> Running diagnostics
#> Running SCC diagnostics
#> - Running MDRR (power) diagnostic
#> - Running pre-exposure gain diagnostic
#> - Running event-dependent observation diagnostic
#> - Running EASE (systematic error) diagnostic
#> Completed 3 diagnostic tests
#> 1 diagnostic test(s) failed:
#> Computing time at risk distribution statistics
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#> Cleaning up intermedate tables
#>   |                                                                      |
#> Performing SCC analysis took 2.28 secs
```

The results file now contains calibrated estimates (`calibrated_rr`, `calibrated_lb_95`, `calibrated_ub_95`, `calibrated_p_value`).

```
results <- read.csv(file.path(outputFolder, "scc_result.csv"))

results %>%
```

```
  filter(analysis_id == 4, target_cohort_id == 4) %>%
  select(target_cohort_id, rr, calibrated_rr, p_value, calibrated_p_value) %>%
  knitr::kable(digits = 3)
```

target_cohort_id	rr	calibrated_rr	p_value	calibrated_p_value

Note: With only 2 negative controls, the calibration will be very approximate. Real studies should use dozens of negative controls.

7 Multi-Analysis Workflow

For larger studies, it is common practice to perform multiple distinct analyses (using different risk window settings) across several exposure-outcome pairs simultaneously. You can use the `runSccAnalyses` function to handle this workflow.

First, define the parameters for each analysis:

```
# Analysis 1: Standard risk windows (1-30 days exposed vs pre-exposure)
sccArgs1 <- createRunSelfControlledCohortArgs(
  riskWindowStartExposed = 1,
  riskWindowEndExposed = 30,
  addLengthOfExposureExposed = FALSE,
  riskWindowStartUnexposed = -30,
  riskWindowEndUnexposed = -1,
  addLengthOfExposureUnexposed = FALSE
)
analysis1 <- createSccAnalysis(
  analysisId = 101,
  description = "Standard 30-day windows",
  runSelfControlledCohortArgs = sccArgs1
)

# Analysis 2: Longer exposure risk window (1-90 days exposed)
sccArgs2 <- createRunSelfControlledCohortArgs(
  riskWindowStartExposed = 1,
  riskWindowEndExposed = 90,
  addLengthOfExposureExposed = FALSE,
  riskWindowStartUnexposed = -90,
  riskWindowEndUnexposed = -1,
  addLengthOfExposureUnexposed = FALSE
)
analysis2 <- createSccAnalysis(
  analysisId = 102,
  description = "Extended 90-day windows",
  runSelfControlledCohortArgs = sccArgs2
)
```

Next, define your exposure-outcome hypotheses:

```
# Create hypotheses (treating IDs 1 and 2 as negative controls)
eo1 <- createExposureOutcome(exposureId = 4, outcomeId = 3) # Target (NSAIDs - GiBleed)
eo2 <- createExposureOutcome(exposureId = 1, outcomeId = 3, trueEffectSize = 1) # Control 1
eo3 <- createExposureOutcome(exposureId = 2, outcomeId = 3, trueEffectSize = 1) # Control 2
```

Finally, execute the analyses spanning all combinations:

```
multiResultsFolder <- tempfile("scc_multi")

runSccAnalyses(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = cdmDatabaseSchema,
  exposureTable = "cohort",
  outcomeTable = "cohort",
  resultsFolder = multiResultsFolder,
  sccAnalysisList = list(analysis1, analysis2),
  exposureOutcomeList = list(eo1, eo2, eo3),
  databaseId = 1,
  computeThreads = 1,
  analysisThreads = 1 # Serial execution for Eunomia SQLite
)
#> *** Running multiple analysis ***
#> Initiating cluster consisting only of main thread
#> Computing time at risk exposed and unexposed windows
#>   |                                                                      |
#> Retrieving counts from database
#>   |                                                                      |
#> Computing raw effect estimates
#> Initiating cluster consisting only of main thread
#> Running diagnostics
#> Running SCC diagnostics
#> - Running MDRR (power) diagnostic
#> - Running pre-exposure gain diagnostic
#> - Running event-dependent observation diagnostic
#> - Running EASE (systematic error) diagnostic
#> Completed 15 diagnostic tests
#> 5 diagnostic test(s) failed:
#> Computing time at risk distribution statistics
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#> Cleaning up intermedate tables
#>   |                                                                      |
#> Performing SCC analysis took 0.571 secs
#> Computing time at risk exposed and unexposed windows
#>   |                                                                      |
#> Retrieving counts from database
#>   |                                                                      |
#> Computing raw effect estimates
#> Initiating cluster consisting only of main thread
#> Running diagnostics
#> Running SCC diagnostics
#> - Running MDRR (power) diagnostic
#> - Running pre-exposure gain diagnostic
#> - Running event-dependent observation diagnostic
#> - Running EASE (systematic error) diagnostic
#> Completed 15 diagnostic tests
```

```
#> 5 diagnostic test(s) failed:
#> Computing time at risk distribution statistics
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#>   |                                                                      |
#> Cleaning up intermedate tables
#>   |                                                                      |
#> Performing SCC analysis took 0.564 secs
```

This generates comprehensive outputs across the entire combinatorics of tests.

8 Working with the Results Database

The `SelfControlledCohort` package provides tools to create a robust results data model. This is useful for sharing results or powering a Shiny application.

In this section, we will: 1. Create a local SQLite database to store our results. 2. Create the standard results schema tables. 3. Upload the CSV results we generated into this database. 4. Query the database to see the results.

First, let's create a new local SQLite database file for the results.

```
resultsDbFile <- tempfile("scc_results_db", fileext = ".sqlite")
resultsConnectionDetails <- createConnectionDetails(dbms = "sqlite", server = resultsDbFile)
```

Next, we use `createResultsDataModel` to create the tables. For SQLite, we must use the 'main' schema.

```
createResultsDataModel(
  connectionDetails = resultsConnectionDetails,
  databaseSchema = "main"
)
#> Migrating data set
#> Migrator using SQL files in SelfControlledCohort
#> Creating migrations table
#>   |                                                                      |
#> Migrations table created
#> Executing migration: Migration_1-2.0.0.sql
#>   |                                                                      |
#> Saving migration: Migration_1-2.0.0.sql
#>   |                                                                      |
#> Migration complete Migration_1-2.0.0.sql
#> Closing database connection
```

Now we upload the results from our `outputFolder` into the database.

```
# For this example, we will upload the main results and settings.
# We remove other tables to simplify the upload for this demonstration.
filesToRemove <- c("scc_diagnostics_summary.csv", "scc_stat.csv", "scc_outcome_exposure.csv")
for (f in filesToRemove) {
  if (file.exists(file.path(outputFolder, f))) {
    unlink(file.path(outputFolder, f))
  }
}
```

```
uploadResults(
  connectionDetails = resultsConnectionDetails,
  schema = "main",
  resultsFolder = outputFolder
)
```

Finally, we can connect to this results database and query it like any other OMOP CDM database.

```
resultsConn <- connect(resultsConnectionDetails)

# Query the scc_result table for our calibrated run (Analysis 4)
resultsDb <- querySql(resultsConn, "SELECT * FROM main.scc_result WHERE analysis_id = 4")

# Show the results from the database
resultsDb %>%
  select(target_cohort_id, outcome_cohort_id, rr, calibrated_rr) %>%
  knitr::kable(digits = 3)
```

target_cohort_id	outcome_cohort_id	rr	calibrated_rr

```
disconnect(resultsConn)
```